

CHECK WHAT YOU LEARNT

Check Yourself - Unit 4**A. Choose the correct answer**

1. An LCD dot produces
(a) its own light (b) no light of its own (c) heat (d) sound
2. The contrast of the display is set by
(a) the backlight (b) the potentiometer on V0 (c) the RS pin (d) the library
3. The HD44780 is
(a) a sensor (b) the display's controller chip (c) a resistor (d) a library
4. 4-bit mode is used in order to
(a) run faster (b) save Arduino pins (c) improve contrast (d) reduce power
5. `setCursor(15, 1)` places the cursor at
(a) row 15, column 1 (b) the last character of the bottom row (c) the first character (d) off screen
6. Rotating screens with `millis()` instead of `delay()` means the program
(a) runs faster (b) never freezes (c) uses less memory (d) needs no library

B. Fill in the blanks

- 1 Every character on the display is drawn on a _____ by _____ grid of dots.
- 2 The RW pin is tied to ground because we only ever _____.
- 3 Solid white blocks on the screen usually mean the _____ is wrong.
- 4 `millis()` reports how long the board has been _____.

C. Answer in one or two lines

- 1 Explain how applying a voltage makes an LCD dot go dark.
- 2 Why does an LCD need a backlight when a phone screen does not?
- 3 Explain what `screen = 1 - screen`; does and why it works.
- 4 Why is a long `delay()` a poor way to rotate between screens?

UNIT 5

Final Project: Line Following Robot

Sessions 12, 13, 14 and 15

Everything so far has stayed on the desk. This one drives itself. Two sensors look at the floor, a driver board handles the power, and a loop that runs many times a second keeps the machine on a painted path.

By the end of this unit you will be able to:

- Explain how an IR sensor distinguishes black from white.
- Explain how an H-bridge reverses a motor.
- Build a closed feedback loop and tune it.
- Debug a system where the machine's own movement changes what it senses.

SESSION 12

one class

Inside an IR Sensor, and Inside a Motor Driver

Today you will

- Learn how an IR pair reads a surface.
- Understand what an H-bridge does.
- Agree the team roles and the battery rules.

An IR sensor module carries two tiny components side by side. One is an **infrared LED** that shines light we cannot see. The other is a **phototransistor** that conducts more when infrared falls on it. Both point at the floor, a few millimetres away.

A white surface scatters that infrared straight back, the phototransistor sees plenty, and the module's comparator outputs LOW. A black surface absorbs almost all of it, the phototransistor sees nothing, and the output goes HIGH. That is the whole mechanism - and it is why the sensor works just as well in a dark room as in a bright one, because it brings its own light.

How an IR sensor tells black from white

It shines invisible infrared light down and watches how much bounces back

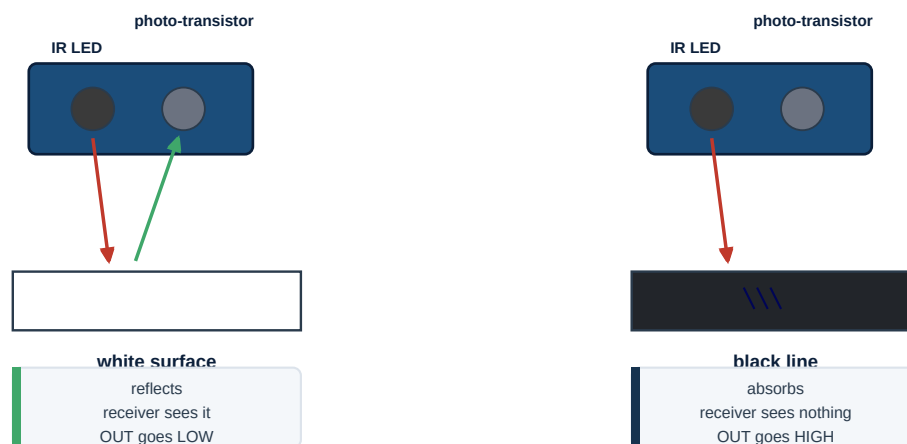


Figure 5.1 - Two tiny components, one very useful trick

Behind the Scenes

The blue trimmer on the module sets the level at which the comparator flips. Too sensitive and every shadow reads as black; too dull and the line disappears. Mounting height matters just as much: about 5 to 10 millimetres above the floor is right. Higher and the reflected light spreads too thin; lower and the sensor sees only its own glow.

12.1 Why a Motor Cannot Touch a Pin

An Arduino output pin can supply about 20 milliamps safely. A BO motor starting under load can demand several hundred - more than ten times as much. Connect one directly and the pin does not merely fail, it burns out permanently.

The **L298N** solves this. Inside are two **H-bridges**, each a set of four electronic switches arranged around a motor in the shape of the letter H. Close one diagonal pair and current flows one way; close the other pair and it flows the other way. The Arduino only ever sends tiny signals telling those switches what to do - it never touches the motor current at all.

The L298N: two switches decide which way a motor turns

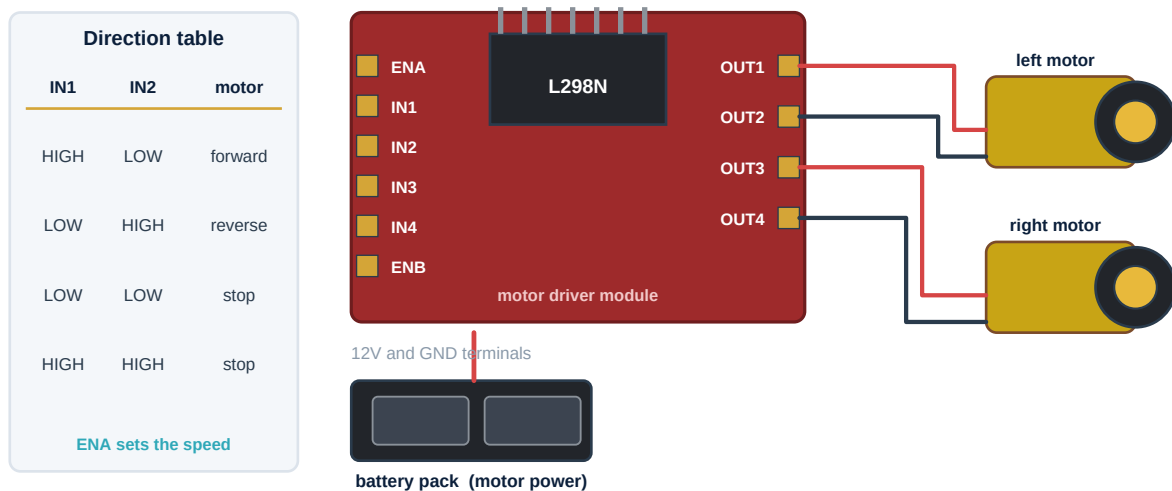


Figure 5.1 - IN pins choose the direction, EN pins choose the speed

12.2 Why the Motor Has a Gearbox

Look closely at your BO motor. The small silver cylinder at the back is the actual motor, and it spins astonishingly fast - several thousand turns a minute - but with almost no strength. Bolted to the front is a **gearbox**, a stack of plastic gears that trades that speed for force.

The bargain is exact. Gear the output down by a factor of fifty and you turn fifty times slower, but with roughly fifty times the turning force. That is why your wheel spins at a sensible 100 revolutions per minute instead of uselessly fast, and why it can actually push a chassis, a battery and a board along the floor.

Quantity	Bare motor	After the gearbox	Why it matters
Speed	very high	about 100 rpm	a fast wheel is uncontrollable
Torque, the turning force	tiny	much larger	it must move the robot's weight
Current at start-up	high	high	this is why a driver board is needed

Watch Out

Two things stop more school robots than any code error. First, the small jumper caps over ENA and ENB: while fitted, the motors run flat out and your speed control does nothing. Remove them. Second, the wire from **Arduino GND to L298N GND**: without it, the two boards do not agree on what zero volts means, and nothing works even though everything looks correct.

Role	Responsible for	Name
Chassis builder	Mounting motors, wheels, caster, battery and sensor height	_____
Wiring engineer	Every connection, checked twice, and the shared ground	_____
Programmer	Typing, uploading and tuning speed and thresholds	_____
Recorder	The notebook: test runs, faults and what fixed them	_____

SESSION 13

one class

Building the Drive Base

Today you will

- Assemble the chassis, motors, wheels and battery.
- Mount the IR sensors at the right height and spacing.
- Test all four motions before adding any sensor logic.

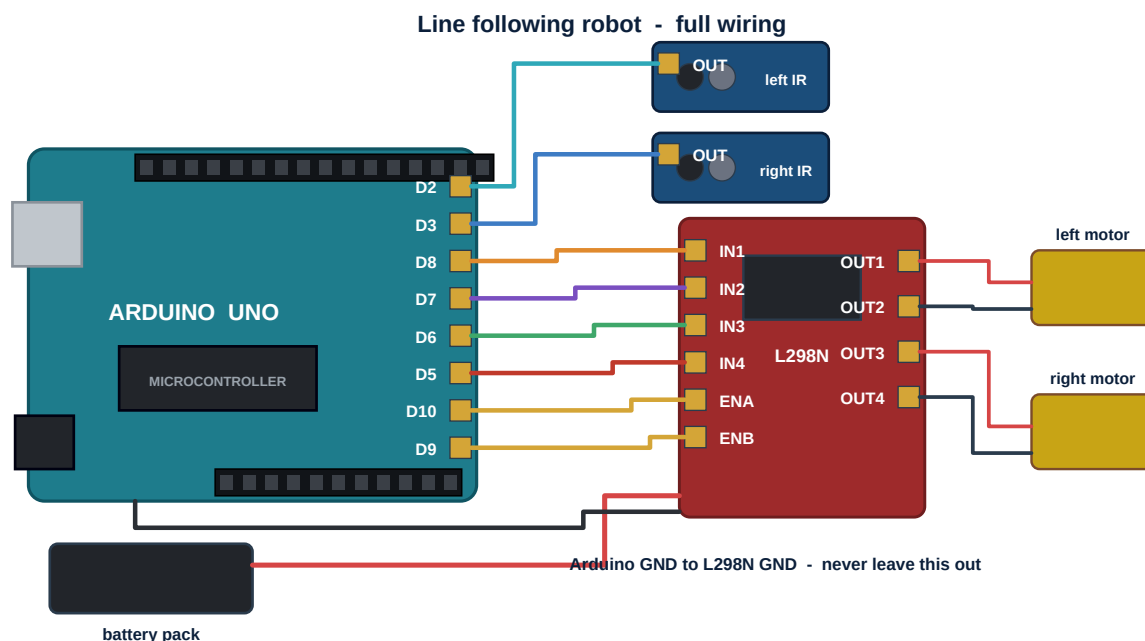


Figure 5.3 - Two eyes, one driver, two wheels

Connection	Goes to	Why
Left IR OUT	D2	Left eye
Right IR OUT	D3	Right eye
L298N IN1 / IN2	D8 / D7	Left motor direction
L298N IN3 / IN4	D6 / D5	Right motor direction
L298N ENA / ENB	D10 / D9	Speed for each motor
L298N OUT1-4	Motor terminals	Power to the wheels
L298N 12V / GND	Battery + / Battery -	Motor power
L298N GND	Arduino GND	Shared ground. Never leave this out.

- Mount the two IR sensors about **2 to 3 centimetres apart**, straddling the line, both pointing straight down.
- Keep them **5 to 10 millimetres** above the floor, and at the very front of the chassis.
- Mount both motors so the wheels are parallel; a few degrees out and the robot curves.
- Keep the battery low and central, and route wires clear of the wheels.

DriveBaseTest.ino

```
int in1 = 8, in2 = 7, in3 = 6, in4 = 5;
int ena = 10, enb = 9;
int speed = 130;           // start slow. 0 to 255.

void setup() {
  pinMode(in1, OUTPUT); pinMode(in2, OUTPUT);
  pinMode(in3, OUTPUT); pinMode(in4, OUTPUT);
  pinMode(ena, OUTPUT); pinMode(enb, OUTPUT);
}
```

```
void forward() {
  analogWrite(ena, speed);  analogWrite(enb, speed);
  digitalWrite(in1, HIGH);  digitalWrite(in2, LOW);
  digitalWrite(in3, HIGH);  digitalWrite(in4, LOW);
}

void left() {               // left wheel slows, right keeps going
  analogWrite(ena, 0);      analogWrite(enb, speed);
  digitalWrite(in3, HIGH);  digitalWrite(in4, LOW);
}

void right() {
  analogWrite(ena, speed);  analogWrite(enb, 0);
  digitalWrite(in1, HIGH);  digitalWrite(in2, LOW);
}

void stopMotors() {
  analogWrite(ena, 0);  analogWrite(enb, 0);
}

void loop() {              // test all four before going further
  forward();  delay(1200);
  left();    delay(600);
  right();   delay(600);
  stopMotors(); delay(1500);
}
```

Watch Out

If a wheel spins the wrong way, do not rewrite the code - swap that motor's two wires at the L298N terminals. It is faster, and it is what a professional would do.

SESSION 14

one class

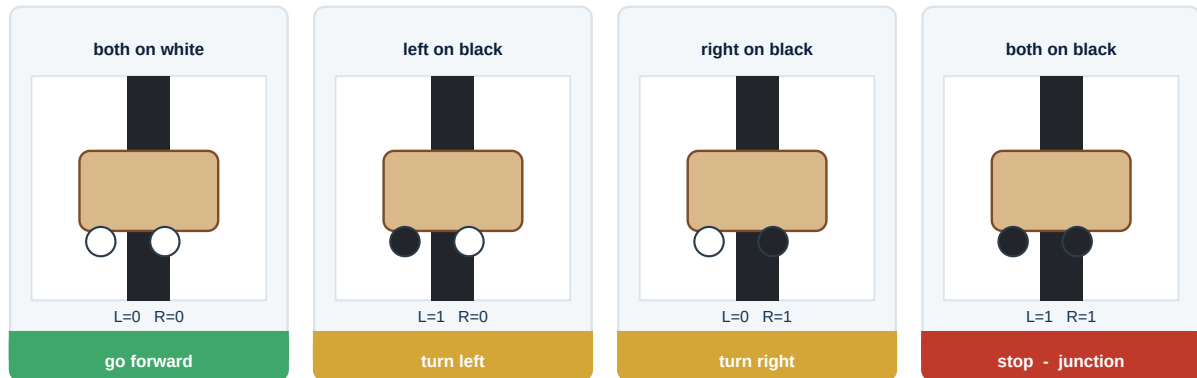
The Four-Case Algorithm

Today you will

- Work out every combination the two sensors can produce.
- Write the line-following program.
- See a closed feedback loop working for the first time.

Two sensors, each with two possible states, gives exactly four combinations - and that is the entire algorithm. Write them down before you write any code.

Four cases, and that is the entire line-following algorithm



A filled circle means that sensor is over black. HIGH means black, LOW means white.

Figure 5.2 - A closed feedback loop: the robot's own moves change what it sees next

What makes this genuinely different from everything before is that it is a **closed feedback loop**. The robot's own movement changes what the sensors see next, which changes the next movement. Nothing else in this book behaves that way, and it is why tuning matters so much: too fast and the robot overshoots before it can correct, too slow and it crawls.

LineFollowingRobot.ino

```
int leftIR = 2, rightIR = 3;
int in1 = 8, in2 = 7, in3 = 6, in4 = 5;
int ena = 10, enb = 9;
int speed = 130;

void setup() {
  pinMode(leftIR, INPUT);
  pinMode(rightIR, INPUT);
  pinMode(in1, OUTPUT); pinMode(in2, OUTPUT);
  pinMode(in3, OUTPUT); pinMode(in4, OUTPUT);
  pinMode(ena, OUTPUT); pinMode(enb, OUTPUT);
  Serial.begin(9600);
  delay(2000);          // time to place the robot and step back
}

void forward() {
  analogWrite(ena, speed); analogWrite(enb, speed);
  digitalWrite(in1, HIGH); digitalWrite(in2, LOW);
  digitalWrite(in3, HIGH); digitalWrite(in4, LOW);
}

void turnLeft() {
  analogWrite(ena, 0);      analogWrite(enb, speed);
  digitalWrite(in3, HIGH); digitalWrite(in4, LOW);
}

void turnRight() {
  analogWrite(ena, speed); analogWrite(enb, 0);
  digitalWrite(in1, HIGH); digitalWrite(in2, LOW);
}

void stopMotors() {
  analogWrite(ena, 0); analogWrite(enb, 0);
}

void loop() {
```

```

int L = digitalRead(leftIR);    // HIGH means black
int R = digitalRead(rightIR);

Serial.print(L); Serial.print(" "); Serial.println(R);

if (L == LOW && R == LOW) {
    forward();                  // both on white - on track
} else if (L == HIGH && R == LOW) {
    turnLeft();                 // drifting right - steer back
} else if (L == LOW && R == HIGH) {
    turnRight();                // drifting left - steer back
} else {
    stopMotors();               // both black - a junction or the end
}

delay(10);                     // a short loop keeps it responsive
}

```

Watch Out

Some IR modules output HIGH on black and others output LOW. Before blaming your algorithm, print L and R and slide a piece of black tape under each sensor by hand. If yours are reversed, swap HIGH and LOW in the four conditions - do not rewire anything.

SESSION 15

one class

Tuning, Track Testing and Your Demonstration**Today you will**

- Tune speed and sensor height on the real track.
- Pass every acceptance test.
- Present your robot in three minutes.

What you see	Most likely cause	What to do
Nothing happens at all	Missing Arduino-to-L298N ground wire	Add it. This is the number one fault.
Motors run flat out, speed ignored	ENA and ENB jumper caps still fitted	Remove both caps
Robot wobbles violently along the line	Speed too high for the correction rate	Lower speed to about 110 and retest
Robot leaves the line on curves	Sensors too close together, or too far apart	Adjust spacing to straddle the line snugly
Reacts to shadows	Trimmer set too sensitive	Turn the blue screw slightly and retest
Stops randomly on a straight	Both sensors briefly reading black	Check sensor height and tape width

Acceptance test	Result	Tick
Follows a 4 metre track with two curves, no rescue	_____	■
Recovers when placed slightly off the line	_____	■
Stops at a junction where both sensors see black	_____	■
Three consecutive runs with no failure	_____	■
Runs for 3 minutes without resetting	_____	■

15.1 Beyond Four Cases: Steering Proportionally

Your robot steers with only three answers: straight, hard left, hard right. That is why it wobbles. A real machine does something smarter - it steers **in proportion** to how far off course it is. A small drift gets a small correction; a large drift gets a large one.

With two digital sensors you cannot measure *how far* off you are, only *whether* you are. But the modules also have an AO pin. Read those on A0 and A1, subtract one from the other, and you get an **error** value: near zero when centred, large and positive when drifting one way, large and negative the other. Then set each motor's speed from that error.

Proportional steering - the idea in eight lines

```
int error = analogRead(A0) - analogRead(A1); // how far off centre, and which way
int correction = error / 6; // 6 is your tuning number

int leftSpeed = speed - correction;
int rightSpeed = speed + correction;

leftSpeed = constrain(leftSpeed, 0, 255); // never go outside 0 to 255
rightSpeed = constrain(rightSpeed, 0, 255);

analogWrite(ena, leftSpeed);
analogWrite(enb, rightSpeed);
```

This is the first half of something called a **PID controller**, which is used in everything from aircraft autopilots to the temperature control of an industrial oven. You have just met the P. It is worth trying even if it takes a whole session to tune.

15.2 Your Three-Minute Demonstration

- The problem** - one sentence on where line-following robots are actually used.
- How it senses** - hold up an IR module and explain the LED and phototransistor pair.
- How it decides** - show the four-case table and explain one case fully.
- How it acts** - explain the H-bridge in one sentence, then run the robot live.
- What went wrong** - name one fault you found and fixed. Judges value this most.
- What next** - one honest improvement you would make with more time.

Challenge

Add a third IR sensor in the centre. With three sensors you can tell a gentle drift from a sharp turn, and steer proportionally instead of jerking. Then add the flame sensor to the front so the robot stops if it meets a fire - a genuine warehouse safety feature.

New word	What it means
Phototransistor	A component that conducts more when light falls on it.
Comparator	A circuit that flips its output when an input crosses a set level.
H-bridge	Four switches that let a circuit reverse a motor on command.
Shared ground	The wire letting two supplies agree on what zero volts means.
Closed feedback loop	A system whose own output changes its next input.